

API 文件前置作業

目錄

- 1.1 API 認證
- 1.2 狀態碼
- 1.3 轉真人 Webhook 事件

API 認證

MaiAgent 使用 API 金鑰進行身分驗證。

身分認證方式

所有 API 請求必須在 HTTP Header 中包含 API Key，格式如下：

```
const headers = {
  "Authorization": "Api-Key YOUR_API_KEY_HERE",
  "Content-Type": "application/json"
}
```

如何獲取 API 金鑰？

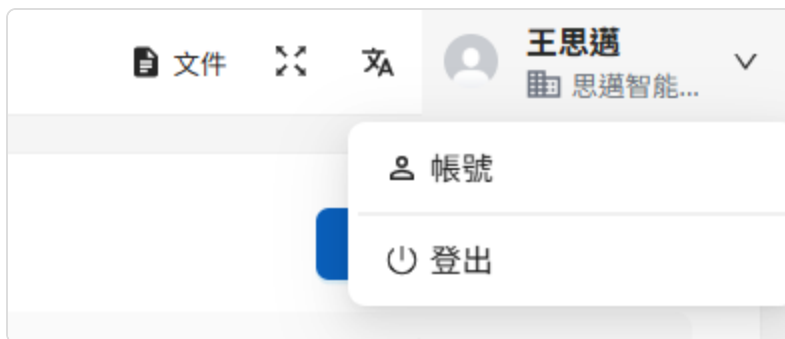
請登入 MaiAgent 系統後台，然後按照以下步驟操作：

點擊右上角的「使用者名稱」下拉選單

點擊「帳號」進入個人資料頁面

The screenshot shows the MaiAgent system interface. The user '王思涵' is logged in. The page displays a table of AI Assistants with columns for ID, Name, Large Language Model, Response Mode, Associated Knowledge Base, Last Update Time, and Actions. Three assistants are listed:

ID	名稱	大語言模型	回答模式	關聯知識庫	最後更新時間	操作
1af8b729-cbca-4c07-b685-...	LLM 測試用	Qwen3 32B(groq)	一般(預設)	LLM 測試用的知識庫	2025-09-09 23:27:51	[Icon] [Icon]
da10bf5e-b8c8-4cac-85cf-...	全螢幕嵌入示範	Claude 4 Sonnet	Agent	露營資料庫	2025-08-27 18:01:48	[Icon] [Icon]
68aa8a32-798a-473c-8cc3-...	台股大盤分析	Claude 4 Sonnet	Agent	台股大盤分析的知識庫	2025-08-12 10:40:59	[Icon] [Icon]



將頁面切換至 API 金鑰頁面。



即可複製查看「API 金鑰」。



安全提醒：請妥善保管您的 API 金鑰，不要在公開場所分享 金鑰格式說明：API 金鑰英數字組合

錯誤處理

當認證錯誤時，API 會回傳 401 Unauthorized 的 HTTP 錯誤代碼：

API Key 無效或缺失

```
{
  "detail": "Authentication credentials were not provided."
}
```

修復建議：檢查請求標頭是否包含正確的 *Authorization* 欄位。格式應為：`Authorization: Api-Key YOUR_API_KEY`

API Key 格式錯誤

```
{
  "detail": "Invalid API key format."
}
```

修復建議：確認 API Key 格式正確。檢查是否使用了正確的前綴 `Api-Key` 而非 `Bearer`，並確認 API Key 本身沒有多餘的空格或字元。

狀態碼

在透過 API 送出 HTTP 請求時，可使用以下狀態碼查看 API 請求成功與否：

狀態碼說明

正確代碼

狀態碼	說明
200	請求成功
201	資源成功建立

正確回應範例

此處為內容示範，不同端點的回傳格式內容可能有差異，請以該端點的說明文件參考為主

```
{
  "count": "integer",
  "next": "string|null",
  "previous": "string|null",
  "results": [
    // 結果回傳內容
  ]
}
```

錯誤代碼

狀態碼	說明
400	請求格式錯誤
401	認證失敗
403	權限不足
404	資源不存在
500	伺服器錯誤

錯誤回應範例

400 - 請求錯誤：必填欄位缺失

```
{
  "files": ["This field is required."],
  "filename": ["This field is required."],
  "file": ["This field is required."]
}
```

轉真人 Webhook 事件

當客服入口（收件匣）完成「轉真人」指派時，MaiAgent 會以 HTTPS POST 將一筆 `handoff.created` 事件推送到你設定的端點，讓派工、告警或 CRM 系統自動接手，不需輪詢。

啟用方式

由客戶的管理者或串接人員在後台自助完成，毋需工程協助：

後台 → 設定 → 收件匣 → 選擇收件匣 → 「轉真人」頁籤 → 「通知」分頁
開啟「Webhook 通知（系統串接）」開關，填入接收端網址（僅接受 `https://` 開頭）
（選填但建議）填入簽章密鑰，啟用來源驗證（見下方「簽章驗證」）

Webhook 與通知中心、Email 是三個獨立開關，互不影響；預設關閉、逐收件匣啟用。此通道與收件匣既有的「訊息 Webhook」（訊息收送）互不相干。

事件格式

```
POST <你設定的網址>
Content-Type: application/json
```

```
{
  "event": "handoff.created",
  "conversationId": "6c2b7c1e- ... ",
  "inboxId": "0f9a3d42- ... ",
  "organizationId": "b1d20c77- ... ",
  "assigneeId": "3e8f5a90- ... ",
  "contactId": "a4c1f6b3- ... ",
  "conversationUrl": "https://admin.maiagent.ai/ ... "
}
```

欄位	類型	說明
event	string	固定為 <code>handoff.created</code>
conversationId	string (uuid)	對話識別碼
inboxId	string (uuid)	收件匣識別碼

欄位	類型	說明
organizationId	string (uuid)	組織識別碼
assigneeId	string (uuid)	null
contactId	string (uuid)	null
conversationUrl	string (uri)	後台對話頁連結

Payload 只含識別資訊與後台連結，**不含對話文字內容**；需要對話內容時，請以識別碼呼叫對話與訊息 API 查詢。

簽章驗證（建議啟用）

在後台設定簽章密鑰後，每次推送會帶下列標頭：

```
X-MaiAgent-Signature-256: sha256=
```

值為以密鑰對 **原始 request body bytes** 計算的 HMAC-SHA256 十六進位摘要。驗證時務必使用收到的原始 bytes，不要先解析再重新序列化（欄位順序與空白差異會使摘要不一致）。

```
import hashlib
import hmac

def verify_signature(raw_body: bytes, header_value: str, secret: str) → bool:
    expected = 'sha256=' + hmac.new(secret.encode(), raw_body,
    hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)
```

密鑰為 write-only：儲存後不回顯，後台僅顯示「已設定」；欄位留空儲存代表維持既有密鑰，更換直接輸入新值，移除請使用「清除密鑰」操作。未設定密鑰時，推送不帶簽章標頭。

回應與重試

接收端應在 10 秒內以 2xx 狀態碼回應，回應內容不拘。

逾時、連線失敗或非 2xx 回應時，系統會自動重試，最多 3 次、間隔 60 秒；重試送出的內容與原始推送完全相同，接收端請自行冪等處理（例如以 `conversationId` 去重）。

Webhook 推送失敗不影響轉真人主流程，也不影響通知中心與 Email 通知。

